

Implementación y análisis comparativo de rendimiento de primitivas RSA y ECC

Trabajo Fin de Grado - Ingeniería Informática

Leon Elliott Fuller

Junio de 2026

Tutor: Jesús García Miranda

E.T.S. de Ingenierías Informática y de Telecomunicación

Universidad de Granada

1. Motivación y objetivos
2. Fundamentos: curvas elípticas
3. Protocolos: ECDH y ECDSA
4. Implementación y metodología
5. Resultados: los tres ejes
6. Conclusiones y trabajo futuro

Motivación y objetivos

- RSA (años 70) sigue siendo el pilar de la clave pública, pero su seguridad exige claves **cada vez más grandes**.
- **1985**: Koblitz y Miller proponen, de forma independiente, usar **curvas elípticas** (ECC).
- Misma seguridad con claves **mucho menores** $\Rightarrow$ idóneas para dispositivos limitados y aplicaciones de alto rendimiento.

Pregunta del trabajo

No basta con saber que ECC es “mejor”. ¿Cuánto vale esa ventaja en la práctica, y por qué?

Objetivo y los tres ejes de comparación

Objetivo: implementar ambos criptosistemas en C++17 y compararlos empíricamente con rigor.

Eje 1

RSA vs ECC

a igual nivel de seguridad
(NIST SP 800-57)

Eje 2

Afines vs Jacobianas

efecto de las coordenadas
proyectivas

Eje 3

$\mathbb{F}_p$ vs $\mathbb{F}_{2^m}$

cuerpos primos vs binarios

El valor del trabajo es separar lo que es **álgebra** de lo que es **ingeniería**.

Fundamentos: curvas elípticas

¿Qué es una curva elíptica?

Sobre un cuerpo K (con $\text{char}(K) \neq 2, 3$), el conjunto de soluciones de la **ecuación de Weierstrass**

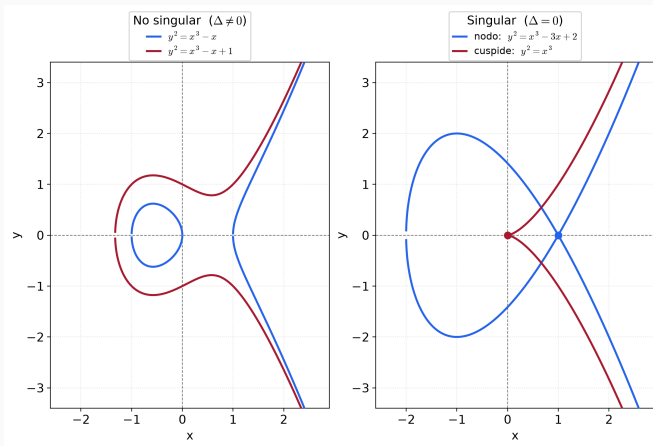
$$y^2 = x^3 + ax + b$$

junto con un **punto del infinito** $\mathcal{O}$.

Condición de no singularidad

$$\Delta = -16(4a^3 + 27b^2) \neq 0$$

(sin cúspides ni autointersecciones).



De la ecuación general a la forma de Weierstrass

Toda curva elíptica parte de la **ecuación general de Weierstrass**:

$$y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6$$

Sobre cuerpos con $\text{char}(K) \neq 2, 3$ se simplifica en dos pasos:

1. **Completar el cuadrado** en y $[y \mapsto y - \frac{1}{2}(a_1x + a_3)]$, **requiere dividir por 2**:

$$y^2 = x^3 + a'_2x^2 + a'_4x + a'_6$$

2. **Eliminar el término cuadrático** $[x \mapsto x - \frac{a'_2}{3}]$, **requiere dividir por 3**:

$$\boxed{y^2 = x^3 + ax + b} \quad (\text{forma corta})$$

Los divisores 2 y 3 son exactamente la razón de exigir $\text{char}(K) \neq 2, 3$.

Curvas sobre cuerpos binarios $\mathbb{F}_{2^m}$

En char 2 **no se puede dividir entre 2**: falla el primer paso. Para curvas **ordinarias** (no supersingulares), un cambio de variable adecuado conduce a la forma que usamos:

$$\boxed{y^2 + xy = x^3 + ax^2 + b}, \quad b \neq 0$$

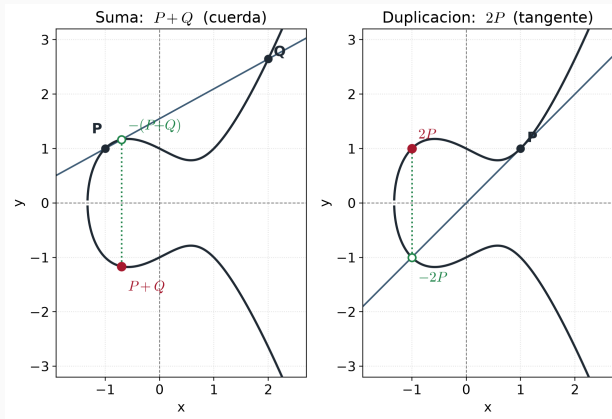
- Es la forma de las curvas binarias **SEC 2** del TFG: sect163k1, sect233k1, sect283k1 (Koblitz) y sect233r1, sect283r1 (aleatorias).
- Curvas de **Koblitz**: $a \in \{0, 1\}$, $b = 1$; admiten optimización por el endomorfismo de Frobenius (no aprovechada en nuestra versión afín).
- No singular $\iff b \neq 0$ (aquí $\Delta = b$).

La ley de grupo: cómo se “suman” puntos

Los puntos forman un **grupo abeliano** $(E(K), +)$:

- **Suma** $P + Q$: la recta por P y Q corta la curva en un tercer punto; su **reflexión** es $P + Q$.
- **Duplicación** $2P$: igual, con la **tangente** en P .
- **Neutro**: $\mathcal{O}$ (rectas verticales).

Es la **operación atómica** que se repite en todo ECC.



El grupo $(E(K), +)$: estructura y orden

Grupo abeliano. Para todo P, Q, R :

- Clausura: $P + Q \in E(K)$
- Asociatividad:
 $(P + Q) + R = P + (Q + R)$
- Neutro: $P + \mathcal{O} = P$
- Inverso: $P + (-P) = \mathcal{O}$
- Conmutatividad: $P + Q = Q + P$

Orden $\#E(\mathbb{F}_q)$ (número de puntos):

Teorema de Hasse

$$|\#E(\mathbb{F}_q) - (q + 1)| \leq 2\sqrt{q}$$

Estructura: $E(\mathbb{F}_q) \cong \mathbb{Z}/n_1 \oplus \mathbb{Z}/n_2$, $n_1 \mid n_2$.

En cripto: subgrupo cíclico de **orden primo** n , con $\#E = h \cdot n$ (cofactor h pequeño). La dificultad del ECDLP es $\sim \sqrt{n}$.

Suma de puntos: fórmulas en $\mathbb{F}_p$ y $\mathbb{F}_{2^m}$

Cuerpo primo $y^2 = x^3 + ax + b$

Suma ($P \neq Q$): $\lambda = \frac{y_2 - y_1}{x_2 - x_1}$

$x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1$

Duplicación ($2P$): $\lambda = \frac{3x_1^2 + a}{2y_1}$

$x_3 = \lambda^2 - 2x_1, \quad y_3 = \lambda(x_1 - x_3) - y_1$

$-P = (x_1, -y_1)$

Cuerpo binario $y^2 + xy = x^3 + ax^2 + b$

Suma ($P \neq Q$): $\lambda = \frac{y_1 + y_2}{x_1 + x_2}$

$x_3 = \lambda^2 + \lambda + x_1 + x_2 + a$

$y_3 = \lambda(x_1 + x_3) + x_3 + y_1$

Duplicación ($2P$): $\lambda = x_1 + \frac{y_1}{x_1}$

$x_3 = \lambda^2 + \lambda + a, \quad y_3 = x_1^2 + (\lambda + 1)x_3$

$-P = (x_1, x_1 + y_1)$; restar = sumar

Ambas requieren una división (inversión en el cuerpo) para calcular λ .

Afines

- Un punto es $(x, y) \in K^2$ sobre la curva, más $\mathcal{O}$.
- Directas, pero cada suma exige una inversión (para λ).

Jacobianas (proyectivas)

- Clase $(X : Y : Z)$ con $x = \frac{X}{Z^2}$, $y = \frac{Y}{Z^3}$.
- $(X : Y : Z) \sim (\lambda^2 X : \lambda^3 Y : \lambda Z)$;
 $\mathcal{O} = (1 : 1 : 0)$.
- Curva: $Y^2 = X^3 + aXZ^4 + bZ^6$.

Al no normalizar (no dividir por Z) se difiere una única inversión al final, en lugar de una por cada operación.

¿Por qué las Jacobianas son más eficientes?

- La **inversión modular** es la operación más cara del cuerpo.
- Modelo clásico: $I \approx 80\text{--}100 M$ (multiplicaciones).
- Recuento de una *duplicación*:
 - Afín: $1I + 2M + 2S$
 - Jacobiana: $4M + 6S$ (**sin I**)
- Si $I \gg M$, ganan las Jacobianas: **una sola inversión al final**.

Conexión con los benchmarks

La teoría predice $\sim 8\times$; nosotros medimos $\sim 1.8\times$.

NTL/GMP hacen la inversión tan rápida que el cociente real I/M es mucho menor que 80–100, y el ahorro se reduce.

Multiplicación escalar y el problema difícil

Multiplicación escalar: $kP = \underbrace{P + P + \dots + P}_{k \text{ veces}}$, calculada con **double-and-add** en $O(\log k)$ pasos.

Fácil

Dados k y P , calcular $Q = kP$.

Inviabile (ECDLP)

Dados P y $Q = kP$, hallar k .

Esta asimetría es el **Problema del Logaritmo Discreto en Curvas Elípticas (ECDLP)**.
Sobre él descansan **ECDH** y **ECDSA**: k es la clave privada, $Q = kP$ la pública.

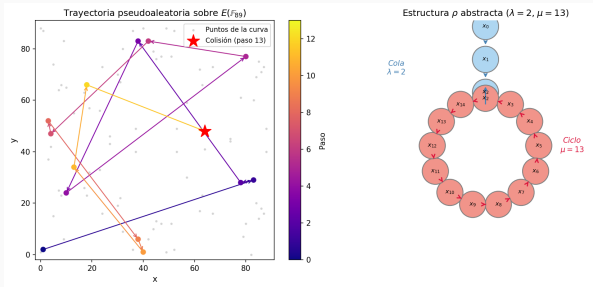
Atacar el ECDLP: el algoritmo rho de Pollard

Mejor ataque **genérico** conocido:

- Paseo **pseudoaleatorio** sobre el grupo de puntos.
- Por la **paradoja del cumpleaños**, aparece pronto una **colisión**: el camino entra en un ciclo (forma de ρ : cola + ciclo).
- Detección de ciclo con poca memoria (Floyd: tortuga/liebre). De la colisión se despeja k .

Coste

$O(\sqrt{n})$ en tiempo, $O(1)$ en memoria $\Rightarrow$ **exponencial** en bits.



La consecuencia: claves mucho más pequeñas

- **RSA**: factorización $\rightarrow$ ataques **subexponenciales** ($L_n[1/3]$).
- **ECC**: ECDLP $\rightarrow$ mejor ataque **exponencial**, $O(\sqrt{n})$.

Idea clave

La ventaja de tamaño de clave es **estructural** y **crece** con el nivel de seguridad.

Seguridad	RSA	ECC	Razón
80 bits	1024 b	160 b	6×
112 bits	2048 b	224 b	9×
128 bits	3072 b	256 b	12×
192 bits	7680 b	384 b	20×
256 bits	15360 b	512 b	30×

Equivalencias NIST SP 800-57. La razón **crece** con la seguridad.

Protocolos: ECDH y ECDSA

Parámetros del dominio

Para instanciar ECC sobre $\mathbb{F}_p$, todas las partes acuerdan la séxtupla

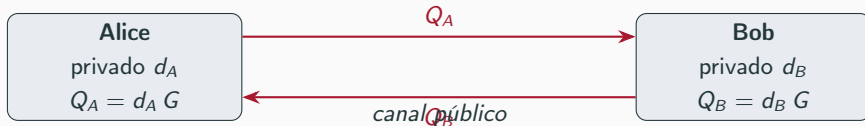
$$T = (p, a, b, G, n, h)$$

Requisitos de seguridad

- p : primo que define $\mathbb{F}_p$.
- a, b : coeficientes de $y^2 = x^3 + ax + b$.
- $G = (G_x, G_y)$: punto generador.
- n : orden de G (menor con $nG = \mathcal{O}$).
- $h = \#E(\mathbb{F}_p)/n$: cofactor.
- n primo (evita Pohlig–Hellman).
- h pequeño, ideal 1 (evita curva inválida).
- No anómala: $\#E(\mathbb{F}_p) \neq p$.
- Grado de inmersión alto (evita MOV).
- $\Delta = -16(4a^3 + 27b^2) \neq 0$.

En el TFG usamos curvas estándar **NIST/SECG**, ya auditadas por la comunidad criptográfica.

Intercambio de claves: ECDH



Ambos calculan el **mismo secreto** (asociatividad y conmutatividad):

$$S = d_A Q_B = d_B Q_A = d_A d_B G \implies \text{clave de sesión} = \text{KDF}(x_S)$$

Un intruso ve G , Q_A , Q_B , pero recuperar d_A o d_B es el ECDLP.

Firma digital: ECDSA

Firma (clave privada d)

1. $e = H(m)$
2. nonce $k \xleftarrow{\$} [1, n - 1]$
3. $R = kG = (x_R, y_R)$, $r = x_R \text{ mód } n$
4. $s = k^{-1}(e + d r) \text{ mód } n$

Firma: (r, s)

Verificación (clave pública $Q = dG$)

1. $e = H(m)$, $w = s^{-1} \text{ mód } n$
2. $u_1 = e w \text{ mód } n$, $u_2 = r w \text{ mód } n$
3. $R' = u_1 G + u_2 Q$
4. aceptar si $x_{R'} \text{ mód } n = r$

El nonce k es crítico

Si k se reutiliza o es predecible, se recupera la clave privada d (casos PlayStation 3 y Bitcoin).
Solución: nonce determinista (RFC 6979).

Implementación y metodología

Arquitectura modular

- RSA con descifrado acelerado por CRT
- ECC afín y Jacobiano sobre $\mathbb{F}_p$
- ECC sobre cuerpos binarios $\mathbb{F}_{2^m}$
- SHA-256 desde cero (FIPS 180-4)
- RNG validado, cinco modos CLI

Stack

C++17

NTL + GMP

Docker (reproducibilidad)

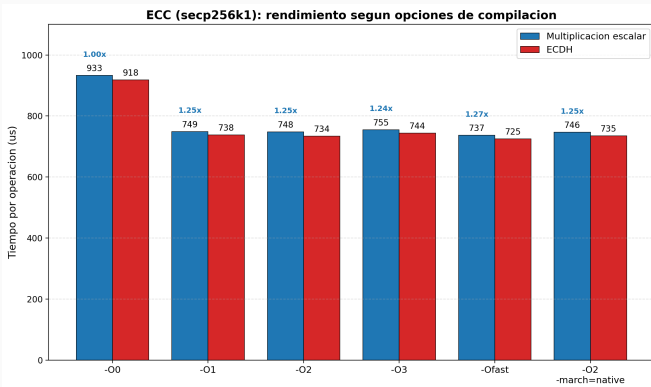
SHA-256 a mano: entender el hash de ECDSA y evitar que librerías externas contaminen los tiempos.

- **Docker**: mismo entorno, reproducible.
- **RNG con semilla fija**: comparación **pareada**, todos ven los mismos datos.
- **Mediana** + calentamiento: robustez frente a outliers y caché fría.

Justicia de la comparación

Se compara a **igual nivel de seguridad** (NIST SP 800-57), *nunca* a igual tamaño de clave.

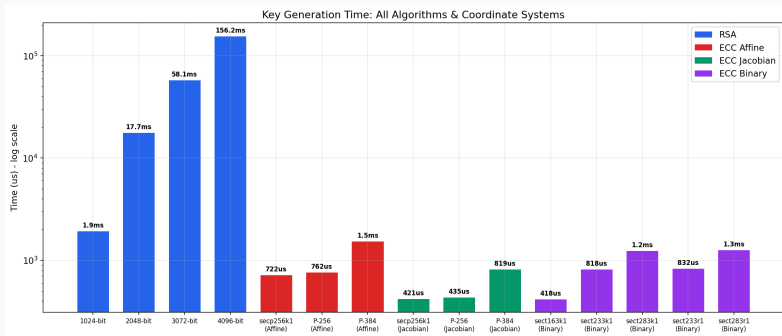
Metodología – elección de las opciones de compilación



De `-O0` a `-O1` se gana $\sim 1.25\times$; por encima de `-O2` las mejoras son marginales (el cálculo pesado vive en GMP, ya precompilado) y `-O3` puede introducir ruido. Se fijó `-O2` como estándar de compilación del TFG.

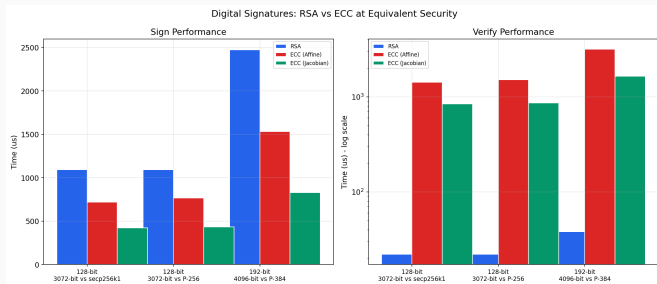
Resultados: los tres ejes

Eje 1 – RSA frente a ECC: generación de claves



RSA debe **buscar primos grandes** por ensayo y error; ECC solo **elige un escalar**. RSA-3072 ≈ 58 ms frente a ECC Jacobiano ≈ 0.42 ms: $\sim 138\times$ (escala logarítmica).

Eje 1 – RSA frente a ECC: firma y verificación

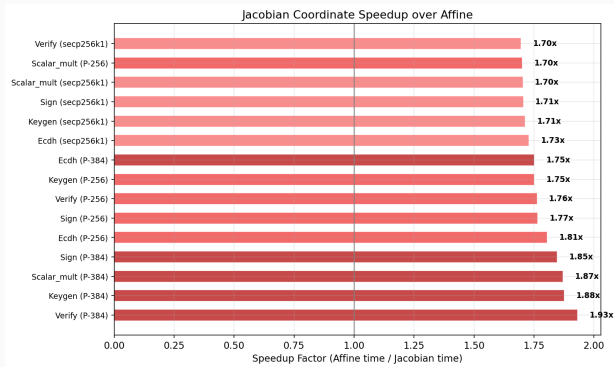


- **Firma:** ECC $\sim 2.6\times$ más rápida.
- **Verificación:** RSA $\sim 38\times$ más rápida.

RSA gana en verificación por su exponente público $e = 65537$ (solo 2 bits a 1).

No hay ganador absoluto: depende del perfil de uso.

Eje 2 – Coordenadas afines vs Jacobianas

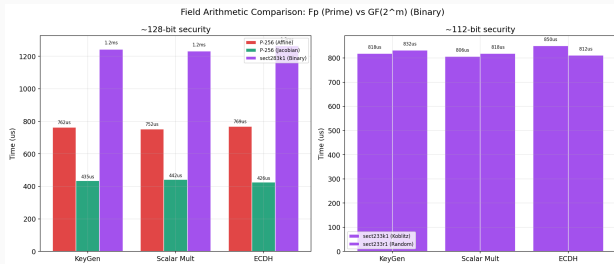


- Las Jacobianas **evitan la inversión modular** (visto en fundamentos).
- Medido: **1.70–1.93×** (media $\sim 1.8\times$).

La brecha ES el hallazgo

La teoría predice $\sim 8\times$; NTL/GMP hacen la inversión barata, y el ahorro real es mucho menor.

Eje 3 – Cuerpos primos vs binarios



- A 128 bits: el binario resulta **$\sim 1.7\times$ más lento** que el primo (afín).

Por qué (honesto):

- GF2E de NTL es genérico vs aritmética sobre ensamblador de GMP.
- El binario solo se midió en **afín**.

Es un **artefacto de software**, no del álgebra.

Validación: contraste con OpenSSL

	OpenSSL vs propia
P-256 (operaciones)	14–34× más rápido
P-384	1.7–4× más rápido
RSA-4096 verificación	a la par (1.08×)

Por qué la diferencia

Ensamblador a mano + **blinding** contra canales laterales que mi versión académica no incorpora.

Objetivo del TFG: **entender y comparar**, no competir con código de producción.

Conclusiones y trabajo futuro

Limitaciones (conscientes, de alcance)

- No es de **tiempo constante** $\Rightarrow$ no apto para producción.
- RSA **de libro**: sin OAEP ni PSS (trabajo futuro).
- El modelo **Jacobiano** solo se implementó para cuerpos primos $\mathbb{F}_p$.

Criterio

Adelantar las limitaciones es integridad académica: son decisiones de alcance documentadas, no olvidos.

Estructural (transferible)

La ventaja de ECC en tamaño de clave es **matemática** (ECDLP exponencial) y **crece** con la seguridad.

De implementación (del entorno)

Las cifras de velocidad (Jacobiano $1.8\times$, binario lento) dependen de compilador, arquitectura y bibliotecas.

El valor del trabajo: separar el álgebra de la ingeniería.

ECC hoy en día: ¿dónde se usa?

- **Web (TLS 1.3):** ~97–98 % de los handshakes usan ECDHE; **X25519** por defecto en navegadores.
- **Passkeys / FIDO2:** firmadas con **ECDSA P-256**.
- **Mensajería y redes:** Signal, WhatsApp, SSH, WireGuard → **Curve25519 / Ed25519**.
- **Criptomonedas:** Bitcoin y Ethereum → **secp256k1** (la curva de Koblitz del TFG).
- **Documentos:** pasaportes electrónicos (ICAO Doc 9303).

La transición post-cuántica es *híbrida*, no una retirada

X25519MLKEM768 (X25519 + ML-KEM-768) ya cubre >40 % del tráfico HTTPS humano en Cloudflare, por defecto en Chrome, Firefox, OpenSSL y Apple. X25519 sigue como **salvaguarda clásica**.

Trabajo futuro

- Padding seguro: **OAEP** (cifrado) y **PSS** (firma).
- Optimizaciones: **NAF**, precomputación; portar el binario a Jacobiano.
- **Horizonte post-cuántico**: Shor rompe **RSA y ECC a la vez**.
NIST (2024): ML-KEM, ML-DSA, SLH-DSA (retículos y hashes).

Recorrido

El banco de pruebas construido es **directamente reutilizable** para comparar los esquemas post-cuánticos frente a ECC.

Gracias por su atención

`github.com/leonfullxr/ECC-Thesis`

Respaldo – corrección de la verificación ECDSA

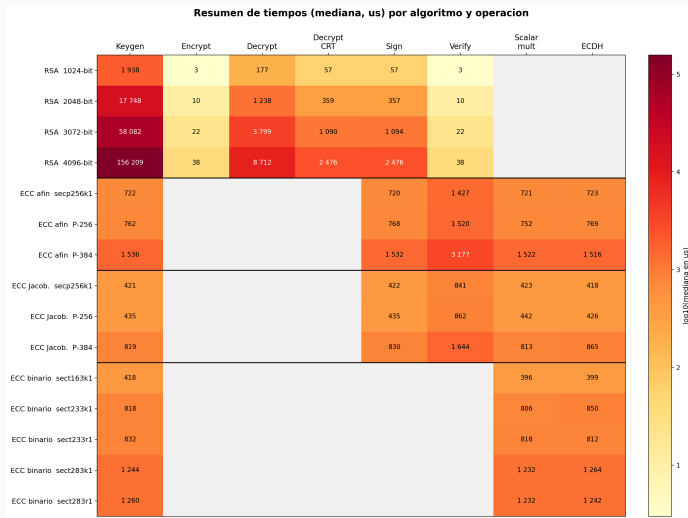
Si la firma es honesta, $s = k^{-1}(e + d r)$ mód n , luego

$$k = s^{-1}(e + d r) = \underbrace{s^{-1}e}_{u_1} + \underbrace{s^{-1}r}_{u_2} d = u_1 + u_2 d \quad (\text{mód } n).$$

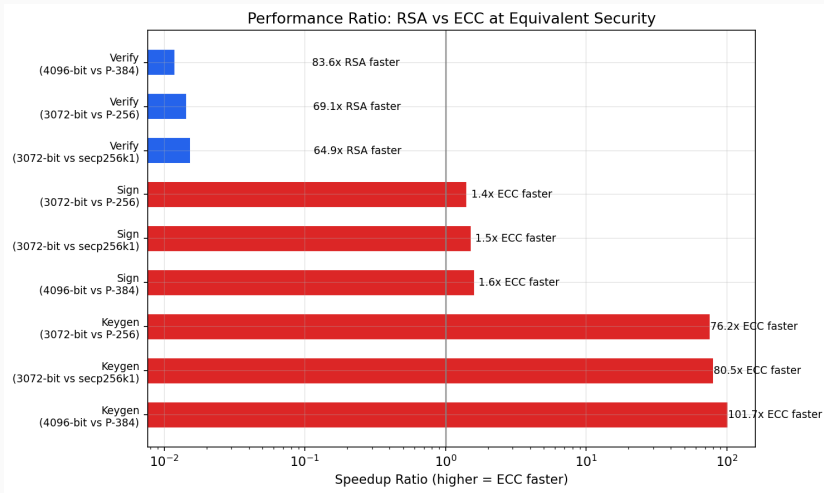
Por tanto $R' = u_1 G + u_2 Q = u_1 G + u_2 d G = (u_1 + u_2 d) G = k G = R$, de donde $x_{R'} \text{ mód } n = x_R \text{ mód } n = r$ y la firma se acepta.



Respaldo – panorama completo de tiempos



Respaldo – razón de rendimiento RSA/ECC



Respaldo – CRT en el descifrado RSA

Se calcula $d_p = d \bmod (p-1)$, $d_q = d \bmod (q-1)$, $q_{inv} = q^{-1} \bmod p$. Recombinación de Garner:
 $m_1 = c^{d_p} \bmod p$, $m_2 = c^{d_q} \bmod q$, $h = q_{inv}(m_1 - m_2) \bmod p$, $m = m_2 + hq$. Acelera $\approx 3.5\times$ (dos exponenciaciones a la mitad de bits).

Respaldo – la asociatividad de la ley de grupo

$(E(K), +)$ es grupo abeliano. Neutro $\mathcal{O}$, inversos (reflexión) y conmutatividad son inmediatos. El axioma no trivial es la **asociatividad**: se prueba con el teorema de Cayley–Bacharach o, más elegantemente, identificando E con su grupo de clases de divisores $\text{Pic}^0(E)$ (Riemann–Roch).